

GABRIEL MAIA MIGUEL

GUY: FERRAMENTA DE ANÁLISE DE  
CÓDIGOS-FONTE PARA GERAR,  
AUTOMATICAMENTE, O GRAFO DE FLUXO DE  
CONTROLE (GFC)

Relatório Técnico elaborado conforme a ABNT NBR 10719:10, apresentado ao Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, como parte dos requisitos para a obtenção do grau de Bacharel em Ciência da Computação.

Área de Concentração: Engenharia de Software

Orientador: Prof. Dr. Breno Lisi Romano

SÃO JOÃO DA BOA VISTA

2026

|                                                                                                                                                                                                                                     |     |            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|------------|
| INSTITUTO FEDERAL DE<br>EDUCAÇÃO, CIÊNCIA E<br>TECNOLOGIA DE SÃO<br>PAULO - CÂMPUS SÃO<br>JOÃO DA BOA VISTA                                                                                                                         | MAR | 2026       |
| <p>Bacharelado em Ciência da Computação</p> <p>GUY: ferramenta de análise de códigos-fonte para<br/>gerar, automaticamente, o Grafo de Fluxo de<br/>Controle (GFC)</p> <p>Gabriel Maia Miguel e<br/>Prof. Dr. Breno Lisi Romano</p> |     |            |
| Palavras-chave:<br>palavra-chave 1. palavra-chave 2. palavra-chave n.                                                                                                                                                               |     | -1 páginas |

Folha destinada à inclusão da Catalogação na Fonte - Ficha Catalográfica (a ser solicitada à Biblioteca IFSP – Câmpus São João da Boa Vista e posteriormente impressa no verso da Folha de Rosto (folha anterior).

Catalogação na Fonte preparada pela Biblioteca Comunitária “Wolgran Junqueira Ferreira”  
do IFSP – Câmpus São João da Boa Vista

Dados da ficha

ATA N.º - DAE-SBV/DRG/SBV/IFSP

**Ata de Defesa de Trabalho de Conclusão de Curso - Graduação**

Na presente data realizou-se a sessão pública de defesa do Trabalho de Conclusão de Curso intitulado  
apresentado(a) pelo(a) aluno(a)  
do  
(Câmpus Câmpus São João da  
Boa Vista). Os trabalhos foram iniciados às pelo(a) Professor(a) presidente da banca examinadora, constituída pelos  
seguintes membros:

| Membros                 | Aprovação |
|-------------------------|-----------|
| (Presidente/Orientador) |           |
| (Examinador 1)          |           |
| (Examinador 2)          |           |

**Observações:**

A banca examinadora, tendo terminado a apresentação do conteúdo da monografia, passou à arguição do(a) candidato(a). Em seguida, os examinadores reuniram-se para avaliação e deram o parecer final sobre o trabalho apresentado pelo(a) aluno(a), tendo sido atribuído o seguinte resultado:

Aprovado(a)                      Reprovado(a)

Proclamados os resultados pelo presidente da banca examinadora, foram encerrados os trabalhos e, para constar, eu lavrei a presente ata que assino juntamente com os demais membros da banca examinadora.

Câmpus São João da Boa Vista,

Avaliador externo:    Sim       Não

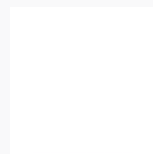
Assinatura:

Documento assinado eletronicamente por:

- PROFESSOR ENS BASICO TECN TECNOLOGICO, em
- PROFESSOR ENS BASICO TECN TECNOLOGICO, em
- PROFESSOR ENS BASICO TECN TECNOLOGICO, em

Este documento foi emitido pelo SUAP em                      Para comprovar sua autenticidade, faça a leitura do QRCode ao lado ou acesse  
<https://suap.ifsp.edu.br/autenticar-documento/> e forneça os dados abaixo:

Código Verificador:  
Código de Autenticação:



ATA N.º - DAE-SBV/DRG/SBV/IFSP

# SUMÁRIO

|            |                                                             |           |
|------------|-------------------------------------------------------------|-----------|
| <b>1</b>   | <b>INTRODUÇÃO</b>                                           | <b>7</b>  |
| <b>1.1</b> | <b>Justificativa</b>                                        | <b>7</b>  |
| <b>1.2</b> | <b>Objetivos</b>                                            | <b>8</b>  |
| 1.2.1      | Objetivo Geral                                              | 8         |
| 1.2.2      | Objetivos Específicos                                       | 8         |
| <b>2</b>   | <b>CONSIDERAÇÕES GERAIS</b>                                 | <b>9</b>  |
| <b>2.1</b> | <b>Engenharia de Software e Qualidade</b>                   | <b>9</b>  |
| <b>2.2</b> | <b>Teste de Software</b>                                    | <b>9</b>  |
| 2.2.1      | Teste Estrutural (Caixa Branca)                             | 10        |
| 2.2.1.1    | Grafo de Fluxo de Controle (GFC)                            | 10        |
| 2.2.1.2    | Complexidade Ciclômática - V(G)                             | 11        |
| 2.2.2      | Cálculo da Complexidade Ciclômática                         | 12        |
| <b>2.3</b> | <b>Análise Estática de Código</b>                           | <b>12</b> |
| 2.3.1      | Processamento Léxico e Sintático                            | 13        |
| <b>2.4</b> | <b>Tecnologias e Linguagens Adotadas</b>                    | <b>13</b> |
| <b>2.5</b> | <b>Trabalhos Correlatos</b>                                 | <b>13</b> |
| 2.5.1      | VEIL ( <i>Vertical Execution-order Informed Layout</i> )    | 14        |
| 2.5.2      | CodeSurfer                                                  | 14        |
| 2.5.3      | Code2Flow                                                   | 14        |
| <b>3</b>   | <b>METODOLOGIA</b>                                          | <b>16</b> |
| <b>3.1</b> | <b>Panorama Geral da Pesquisa</b>                           | <b>16</b> |
| <b>3.2</b> | <b>Etapas para o Desenvolvimento da Ferramenta Proposta</b> | <b>17</b> |
| <b>4</b>   | <b>RESULTADOS</b>                                           | <b>19</b> |
| <b>4.1</b> | <b>Análise das Ferramentas Disponíveis no Mercado</b>       | <b>19</b> |
| 4.1.1      | SonarQube                                                   | 19        |
| 4.1.2      | Codacy                                                      | 20        |
| 4.1.3      | ESLint                                                      | 21        |
| <b>4.2</b> | <b>Levantamento de Requisitos da Ferramenta GUY</b>         | <b>22</b> |
| <b>4.3</b> | <b>Prototipagem e <i>Design</i> da Ferramenta GUY</b>       | <b>23</b> |
| <b>4.4</b> | <b>Implementação da Ferramenta GUY</b>                      | <b>24</b> |
| 4.4.1      | Estrutura geral da extensão                                 | 25        |
| 4.4.2      | Comandos e formas de análise                                | 26        |
| 4.4.3      | Construção do Grafo de Fluxo de Controle                    | 28        |

|            |                                                                  |           |
|------------|------------------------------------------------------------------|-----------|
| 4.4.4      | Métricas e caminhos independentes . . . . .                      | 31        |
| 4.4.5      | Interface de visualização . . . . .                              | 32        |
| 4.4.6      | Critérios de implementação observados . . . . .                  | 34        |
| <b>4.5</b> | <b>Validação da Ferramenta GUY . . . . .</b>                     | <b>36</b> |
| 4.5.1      | Experimento I: Código com Laço e Condicionais . . . . .          | 36        |
| 4.5.2      | Experimento II: Código Condicional Simples . . . . .             | 37        |
| 4.5.3      | Experimento III: Código com Função, Laço e Condicional . . . . . | 38        |
| <b>4.6</b> | <b>Análise Qualitativa da Ferramenta GUY . . . . .</b>           | <b>39</b> |
| <b>5</b>   | <b>CONCLUSÕES E RECOMENDAÇÕES . . . . .</b>                      | <b>40</b> |

## RESUMO

Neste trabalho é apresentada a formatação que deve ser utilizada nos relatórios técnicos a serem submetidos ao final dos cursos de Graduação e Pós-graduação do IFSP câmpus São João da Boa Vista. Leia com atenção este documento. O máximo de palavras para o resumo é 150 (cento e cinquenta).

**Palavras-chave:** palavra-chave 1. palavra-chave 2. palavra-chave 3. palavra-chave n.

# 1 INTRODUÇÃO

Nas últimas décadas, a Engenharia de Software consolidou-se como uma área essencial para o desenvolvimento de sistemas computacionais. Segundo ??) e ??), esses sistemas passaram a exercer papel central em atividades sociais, econômicas e organizacionais.

À medida que as funcionalidades aumentam, também cresce a complexidade do código-fonte. ??) aponta que a complexidade é um dos principais obstáculos à confiabilidade, o que torna insuficiente depender apenas de inspeção manual. Nesse cenário, um dos desafios é verificar se a lógica implementada se comporta conforme o esperado. Como afirma ??), o objetivo do teste não é provar que o software funciona, mas encontrar falhas antes que elas cheguem ao usuário.

As técnicas de teste de caixa branca, ou teste estrutural, contribuem para essa verificação porque analisam a lógica interna do componente. Por meio delas, é possível identificar caminhos de execução, estruturas de decisão e fluxos que devem ser considerados no planejamento dos testes. Essa análise pode ser representada pelo Grafo de Fluxo de Controle (GFC), uma estrutura abstrata em que os nós representam blocos de instruções e as arestas descrevem o fluxo de controle e os desvios lógicos do programa (??).

O GFC também serve de base para métricas estruturais, com destaque para a Complexidade Ciclomática. Introduzida por ??), essa métrica permite mensurar a complexidade lógica de um módulo e estimar o número de caminhos linearmente independentes que podem orientar a definição de casos de teste.

## 1.1 Justificativa

A garantia da qualidade de software tornou-se um fator crítico na indústria de tecnologia. Defeitos em produção não afetam apenas a experiência do usuário; eles também podem gerar prejuízos financeiros e, em sistemas críticos, comprometer a segurança operacional ??).

Estudos sobre redução de defeitos indicam que o custo para corrigir uma falha após a entrega tende a ser maior do que durante o desenvolvimento. Ao mesmo tempo, a complexidade dos sistemas modernos dificulta a detecção manual de erros lógicos.

Diante desse problema, este trabalho propõe o GUY, uma extensão para o VS Code voltada à geração automática de Grafos de Fluxo de Controle e ao cálculo da Complexidade Ciclomática. A proposta busca apoiar analistas, engenheiros e programadores na verificação estrutural de código-fonte.

## 1.2 Objetivos

### 1.2.1 Objetivo Geral

O objetivo desta pesquisa é desenvolver o GUY, uma extensão para o *Visual Studio Code* (VS Code) destinada à análise estrutural de código-fonte e à geração automática do Grafo de Fluxo de Controle (GFC). A ferramenta fornece uma base visual e analítica para auxiliar desenvolvedores na mensuração da complexidade lógica e no planejamento de testes estruturais.

### 1.2.2 Objetivos Específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

- Fundamentar conceitos de Teste de Software, Caixa Branca e a métrica de Complexidade Ciclomática de McCabe;
- Implementar um analisador capaz de identificar estruturas de decisão e repetição no arquivo de código-fonte aberto no editor;
- Desenvolver o algoritmo para mapeamento e renderização visual dos nós e arestas do Grafo de Fluxo de Controle;
- Automatizar o cálculo da complexidade ciclomática ( $V(G)$ ) com base na topologia do grafo gerado;
- Desenvolver uma extensão para o editor VS Code para que o usuário tenha acesso à ferramenta localmente;
- Validar a ferramenta por meio da comparação entre os grafos e valores de complexidade gerados e os resultados esperados para os códigos analisados.

## 2 CONSIDERAÇÕES GERAIS

### 2.1 Engenharia de Software e Qualidade

A Engenharia de Software compreende a aplicação de abordagens sistemáticas, disciplinadas e quantificáveis ao desenvolvimento, à operação e à manutenção de sistemas ??). Como a computação está presente em grande parte das atividades contemporâneas, assegurar a confiabilidade e a estabilidade das aplicações tornou-se uma exigência fundamental, e não apenas um diferencial técnico.

Embora os níveis de complexidade e os riscos envolvidos variem conforme o domínio, a aplicação de princípios de engenharia permanece indispensável. Essa necessidade é evidenciada em sistemas de segurança crítica, nos quais falhas de software podem resultar em consequências catastróficas. Um exemplo histórico notável é o caso do acelerador linear Therac-25, cujos erros de programação causaram acidentes fatais por superexposição à radiação ??).

A relevância da qualidade, porém, não se restringe a cenários de alto risco. Mesmo que um sistema de gestão de uma padaria não tenha a mesma complexidade ou criticidade de um sistema bancário, médico ou militar, ambos precisam ser confiáveis e sustentáveis ao longo do tempo. Em qualquer escala, o software deve preservar a experiência do usuário e a integridade dos processos que apoia.

A manutenção de padrões de qualidade em projetos colaborativos, portanto, torna-se um desafio crescente. À medida que um sistema evolui, a complexidade técnica e a densidade do código também aumentam, tornando processos de verificação exclusivamente manuais pouco viáveis e sujeitos a falhas humanas.

### 2.2 Teste de Software

O teste de software busca, por meio de um conjunto finito de casos, verificar e validar (V&V) se a aplicação atende aos requisitos esperados ??). Essa etapa representa um investimento significativo: cerca de 50% do tempo e mais de 50% do custo total do desenvolvimento de um sistema são consumidos pela atividade de testes ??).

Esse processo inicia-se em componentes individuais ou pequenos módulos, com o objetivo de identificar falhas lógicas e de dados. Após essa validação inicial, os componentes são integrados ao sistema para testes de ordem superior, cujo objetivo é assegurar que o software atenda aos requisitos do cliente ??). ??) destaca que o teste não deve ser entendido como uma forma de demonstrar que o software funciona, mas como um processo

de executá-lo com a intenção explícita de encontrar falhas. Nessa perspectiva, um “teste bem-sucedido” é aquele que revela um erro ainda não descoberto. Em sistemas cada vez mais complexos, aplicar essa abordagem apenas de forma manual torna-se exaustivo e sujeito a vieses, reforçando a necessidade de apoio automatizado.

### 2.2.1 Teste Estrutural (Caixa Branca)

O teste estrutural, também conhecido como teste de caixa branca, concentra-se na análise da lógica interna do programa (??). Ao contrário do teste de caixa preta, que observa se entradas e saídas correspondem ao esperado, essa técnica exige acesso ao código-fonte para avaliar caminhos de execução, *loops* e condições lógicas.

Sua relevância reside na capacidade de orientar a verificação das estruturas internas do programa. Isso inclui apoiar a identificação de instruções, desvios lógicos (verdadeiro e falso), laços de repetição e caminhos independentes que devem ser considerados durante o planejamento dos testes.

#### 2.2.1.1 Grafo de Fluxo de Controle (GFC)

O Grafo de Fluxo de Controle (GFC) é uma representação gráfica usada na análise estrutural. Ele modela a lógica do programa como um grafo dirigido, em que os **nós** representam blocos de processamento sequencial, sem desvios, e as **arestas** representam o fluxo de controle entre esses nós.

Entre os elementos do GFC estão os nós de predicado, que correspondem a pontos de decisão no código, como **if** ou **while**, dos quais partem múltiplas arestas. Ferramentas de teste e análise estática utilizam o GFC para calcular caminhos independentes e determinar a complexidade lógica do algoritmo, transformando o código textual em uma estrutura matemática navegável.

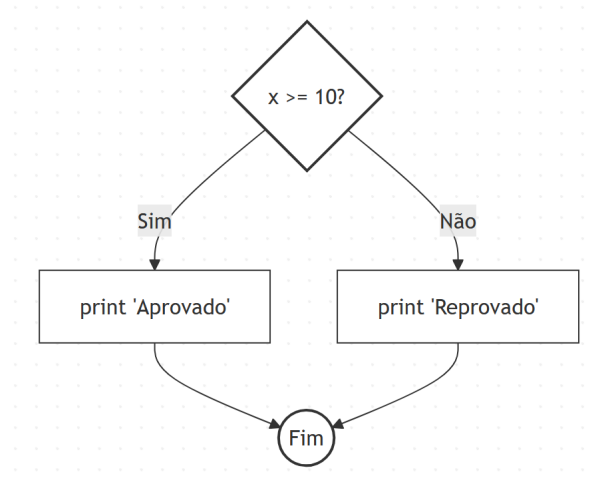
Para consolidar o entendimento sobre a relação entre o código-fonte, o Grafo de Fluxo de Controle (GFC) e a Complexidade Ciclomática, apresenta-se a seguir um algoritmo simples de verificação condicional.

Listing 2.1 – Algoritmo de verificação condicional.

```
1 void VerificaStatus(int x) {  
2     if (x >= 10) {           // Nó de decisão (Predicado)  
3         print("Aprovado");   // Caminho A  
4     } else {  
5         print("Reprovado");  // Caminho B  
6     }  
7     print("Fim");           // Nó de junção/fim  
8 }
```

A partir da análise do Código ??, é possível mapear a lógica para uma representação visual. A Figura ?? ilustra o GFC correspondente, onde os blocos de comando tornam-se nós e o fluxo de execução torna-se arestas direcionadas.

Figura 1 – Exemplo de GFC.



Fonte: Elaborada pelo autor.

### 2.2.1.2 Complexidade Ciclomática - $V(G)$

A Complexidade Ciclomática é uma métrica de software desenvolvida por Thomas McCabe em 1976. Ela quantifica a complexidade lógica de um programa a partir do número de caminhos linearmente independentes em seu Grafo de Fluxo de Controle ??).

A métrica é calculada pela fórmula  $V(G) = E - N + 2P$ , em que  $E$  é o número de arestas,  $N$  é o número de nós e  $P$  é o número de componentes conexos do grafo. Em grafos com apenas um componente conexo, essa expressão simplifica-se para  $V(G) = E - N + 2$ . O valor resultante  $V(G)$  indica o número mínimo de casos de teste necessários para cobrir todos os caminhos independentes do código. McCabe sugere que módulos com

complexidade superior a 10 são mais propensos a erros e difíceis de manter, justificando o uso dessa métrica como um indicador crítico de qualidade e testabilidade ??).

### 2.2.2 Cálculo da Complexidade Ciclomática

Com base na representação visual da Figura ??, é possível aplicar a métrica de McCabe para determinar a complexidade lógica  $V(G)$ . Observando o grafo, identificam-se os seguintes elementos:

- **Nós ( $N$ ):** 4 (Início/If, Print Aprovado, Print Reprovado, Fim).
- **Arestas ( $E$ ):** 4 (As conexões entre os nós).
- **Componentes conexos ( $P$ ):** 1 (O grafo é único e interligado).

Aplicando a fórmula fundamental de McCabe:

$$V(G) = E - N + 2P \quad (2.1)$$

Substituindo pelos valores extraídos do grafo:

$$V(G) = 4 - 4 + 2(1) = 2$$

O resultado  $V(G) = 2$  indica que existem dois caminhos linearmente independentes no código: (1) o caminho em que a condição é verdadeira e (2) o caminho em que a condição é falsa. Esse valor confirma a baixa complexidade do algoritmo e indica que são necessários, no mínimo, dois casos de teste para cobrir a lógica da função.

## 2.3 Análise Estática de Código

A análise estática de código consiste no exame do software sem executá-lo. Essa técnica é adequada para a inspeção estrutural, pois permite verificar a base de código em busca de padrões sintáticos, violações de regras e métricas de complexidade ainda durante o desenvolvimento.

Diferencia-se da análise dinâmica, que requer a execução do programa com dados de teste reais para observar comportamentos em tempo de execução (como uso de memória e falhas). Aplicações práticas da análise estática incluem a detecção precoce de bugs, verificação de conformidade com guias de estilo (linting) e o cálculo automático de métricas como a complexidade ciclomática, permitindo feedback imediato ao desenvolvedor.

### 2.3.1 Processamento Léxico e Sintático

Para construir o GFC e realizar a análise estática, o GUY utiliza conceitos de compiladores. O processo inicia-se com a **tokenização** (análise léxica), que divide o código-fonte em unidades mínimas de significado (tokens) ??).

Em seguida, realiza-se a identificação de estruturas através da **análise sintática**, que organiza esses *tokens* em uma Árvore Sintática Abstrata (AST). A AST é uma representação hierárquica que expõe a estrutura gramatical do código ??). É percorrendo a AST que a ferramenta identifica onde começam e terminam os blocos lógicos (funções, loops, condicionais) para, a partir disso, desenhar os nós e arestas do GFC e calcular a complexidade.

## 2.4 Tecnologias e Linguagens Adotadas

O GUY adota uma arquitetura de extensão para o *Visual Studio Code* (VS Code). A lógica principal executa no *Extension Host*, utilizando Node.js e TypeScript para acessar o arquivo aberto no editor, acionar a análise estática e coordenar a comunicação com a interface. Na camada visual, o framework **React.js** é utilizado em uma *Webview*, permitindo a renderização interativa do Grafo de Fluxo de Controle ??).

Para o processamento de código e análise sintática (*parsing*), a arquitetura foi projetada visando a extensibilidade e o suporte a múltiplos paradigmas de programação, com foco inicial em código Python e possibilidade de expansão futura para outras linguagens. Para isso, adota-se preferencialmente o **Tree-sitter** ??), uma ferramenta moderna de geração de analisadores que pode ser utilizada com *WebAssembly* para executar com desempenho adequado no ambiente local da extensão. Como alternativa consolidada no meio acadêmico e na construção de compiladores, considera-se também a utilização do **ANTLR** (*Another Tool for Language Recognition*) ??).

Essa abordagem agnóstica à linguagem permite que a ferramenta gere a Árvore Sintática Abstrata (AST) e, conseqüentemente, o Grafo de Fluxo de Controle (GFC) para diversas linguagens apenas alternando as gramáticas utilizadas, sem necessidade de reestruturar o núcleo de visualização. Para a renderização visual dos grafos gerados, empregam-se bibliotecas especializadas como **Cytoscape.js** ??) ou **D3.js** ??), garantindo interatividade na *Webview* da extensão.

## 2.5 Trabalhos Correlatos

A pesquisa bibliográfica identificou ferramentas e algoritmos que, assim como a proposta deste trabalho, exploram a representação gráfica de programas para auxiliar na compreensão e no teste de software. Abaixo, destacam-se três abordagens relevantes: um

algoritmo de layout de ponta focado na legibilidade de fluxo (VEIL), uma ferramenta comercial de documentação (Code2Flow) e uma solução industrial de análise profunda (CodeSurfer).

### 2.5.1 VEIL (*Vertical Execution-order Informed Layout*)

O VEIL é um algoritmo de desenho de grafos proposto recentemente por ??), desenvolvido para alinhar a visualização de Grafos de Fluxo de Controle (GFC) com a estrutura mental de leitura de código-fonte. A abordagem utiliza análise de dominadores para aplicar dois novos critérios estéticos: o *Happens-Before* (C10), que garante que nós executados posteriormente sejam desenhados verticalmente abaixo de seus predecessores; e o *Edge Direction Grouping* (C11), que agrupa arestas de retorno (loops) à esquerda e desvios condicionais à direita do grafo ??).

A principal distinção está no escopo. O VEIL é um algoritmo de otimização de layout visual para compiladores e ferramentas de análise, focado em "como desenhar" o grafo para maximizar a legibilidade. O GUY, por outro lado, utiliza a visualização do GFC como apoio ao estudo de métricas de teste, como a Complexidade Ciclomática, permitindo que o estudante interaja com o código e observe o cálculo da métrica no próprio ambiente de desenvolvimento.

### 2.5.2 CodeSurfer

O CodeSurfer, desenvolvido pela GrammaTech, é uma ferramenta de alta precisão para análise estática e navegação em código-fonte complexo (C/C++) ??). Diferente de IDEs comuns, ele constrói uma rede de dependências completa do software, permitindo que o desenvolvedor visualize e navegue pelo Grafo de Fluxo de Controle e pelo Grafo de Dependência de Dados de qualquer função.

A distinção em relação ao trabalho proposto é o público-alvo e a complexidade. O CodeSurfer é uma solução industrial voltada para auditoria de segurança e manutenção de sistemas legados críticos. A ferramenta aqui desenvolvida busca simplificar esse poder de análise, trazendo a geração automática de GFCs para o contexto educacional e para o ambiente local do VS Code, removendo a barreira de entrada e a complexidade de configuração de ferramentas comerciais pesadas.

### 2.5.3 Code2Flow

O Code2Flow é uma ferramenta comercial amplamente utilizada para converter pseudocódigo e códigos de diversas linguagens em fluxogramas interativos ??). Seu foco principal é a documentação de processos e a comunicação lógica entre gerentes e desenvolvedores, permitindo a criação rápida de diagramas a partir de texto.

Apesar da qualidade da geração visual, o Code2Flow foca na representação de processos de negócio e lógica geral, muitas vezes abstraindo detalhes técnicos necessários ao teste estrutural. Diferente do GUY, ele não calcula explicitamente a Complexidade Ciclométrica nem se concentra no ensino de critérios de cobertura de teste, servindo mais como ferramenta de diagramação do que como apoio à análise estrutural de código.

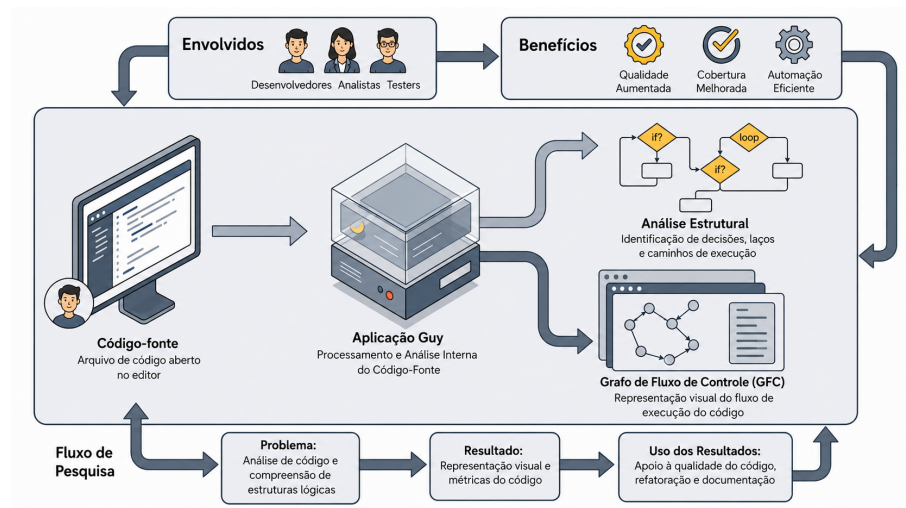
## 3 METODOLOGIA

Este capítulo apresenta a metodologia utilizada no desenvolvimento da extensão GUY, desde a definição do escopo da pesquisa até a implementação e validação da ferramenta.

### 3.1 Panorama Geral da Pesquisa

Como ilustrado na Figura ??, o GUY permite que programadores gerem Grafos de Fluxo de Controle (GFCs) a partir do arquivo aberto no *Visual Studio Code* (VS Code). Com isso, a extensão apoia a compreensão da lógica interna do código e o planejamento de testes estruturais durante o desenvolvimento.

Figura 2 – Fluxo de funcionamento da ferramenta proposta.



Fonte: Elaborada pelo autor (2025).

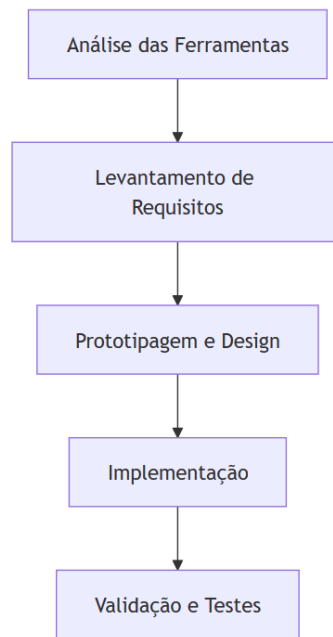
A ferramenta é acionada a partir do arquivo de código-fonte aberto no editor. Em seguida, o núcleo de processamento do GUY executa a análise estática local, mapeia as estruturas lógicas do algoritmo e constrói a representação visual do Grafo de Fluxo de Controle. Essa representação permite identificar os nós de decisão e os caminhos de execução possíveis no trecho analisado.

Com o GFC estruturado, o desenvolvedor passa a visualizar os caminhos independentes do algoritmo. Essa visualização integrada auxilia desenvolvedores e analistas de qualidade (*testers*) na identificação dos fluxos que podem orientar a elaboração de casos de teste estruturais.

## 3.2 Etapas para o Desenvolvimento da Ferramenta Proposta

O desenvolvimento do GUY seguiu uma abordagem de pesquisa aplicada, de caráter exploratório e descritivo, associada à construção experimental de uma ferramenta de apoio à análise estrutural de código. As principais fases do projeto estão sumarizadas na Figura ??.

Figura 3 – Fluxo das etapas metodológicas para o desenvolvimento.



Fonte: Elaborada pelo autor.

A metodologia relaciona fundamentação teórica, levantamento de requisitos, concepção da solução, implementação e validação. O fluxo apresentado na Figura ?? inicia-se pela investigação de ferramentas correlatas, passa pela definição dos requisitos e da arquitetura da solução e termina com a codificação e a avaliação das funcionalidades.

A análise das ferramentas existentes permitiu identificar limitações que serviram de base para os requisitos do GUY. Em seguida, a prototipagem orientou a implementação, e os resultados foram avaliados por meio da comparação entre os grafos e valores de complexidade gerados pela ferramenta e os resultados esperados para os exemplos analisados. Cada bloco apresentado na Figura ?? representa uma etapa do projeto, detalhada a seguir:

1. **Análise das Ferramentas Disponíveis no Mercado:** foram analisadas soluções comerciais e *open source* similares, considerando funcionalidades, custos, usabilidade e limitações em relação à visualização de GFCs.

2. **Levantamento de Requisitos:** foram definidos os requisitos funcionais e não funcionais necessários para a extensão, incluindo execução local, geração do grafo, cálculo de métricas e integração com o editor.
3. **Prototipagem e *Design*:** foram elaborados os *wireframes* e a organização da interface, considerando a visualização do grafo em uma janela lateral do VS Code.
4. **Implementação (Codificação):** a extensão foi desenvolvida com Node.js e TypeScript no *Extension Host* do VS Code, utilizando React para a interface gráfica interativa em *Webview*.
5. **Validação e Testes:** foram realizadas verificações funcionais e comparações entre os resultados gerados pela ferramenta e os grafos e valores de complexidade esperados para os códigos analisados.

Os resultados obtidos são apresentados no Capítulo 4, com o detalhamento da implementação do GUY e da validação das funcionalidades desenvolvidas.

## 4 RESULTADOS

### 4.1 Análise das Ferramentas Disponíveis no Mercado

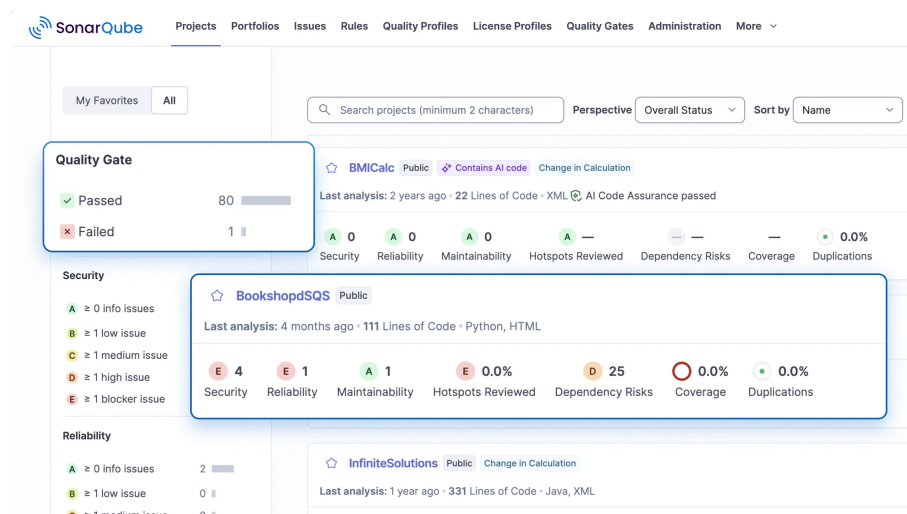
Existem diversas ferramentas voltadas à qualidade de código, mas a maior parte delas não tem como foco a visualização educacional de grafos.

Esta seção apresenta três ferramentas utilizadas na análise de código e discute suas funcionalidades, limitações e diferenças em relação ao GUY.

#### 4.1.1 SonarQube

A ferramenta SonarQube (Figura ??) é uma plataforma de código aberto (*open source*) voltada à inspeção contínua da qualidade do código. Em vez de apenas listar erros, ela permite analisar, rastrear e gerenciar a qualidade do software ao longo do tempo ??).

Figura 4 – Captura de tela da interface do SonarQube.



Fonte: Adaptado de SonarSource (2026).

A plataforma exibe potenciais *bugs*, violações de convenções e boas práticas, além de medir cobertura de testes, duplicação de código e complexidade ciclomática. Também oferece *dashboards*, histórico de análises e integração com IDEs ??).

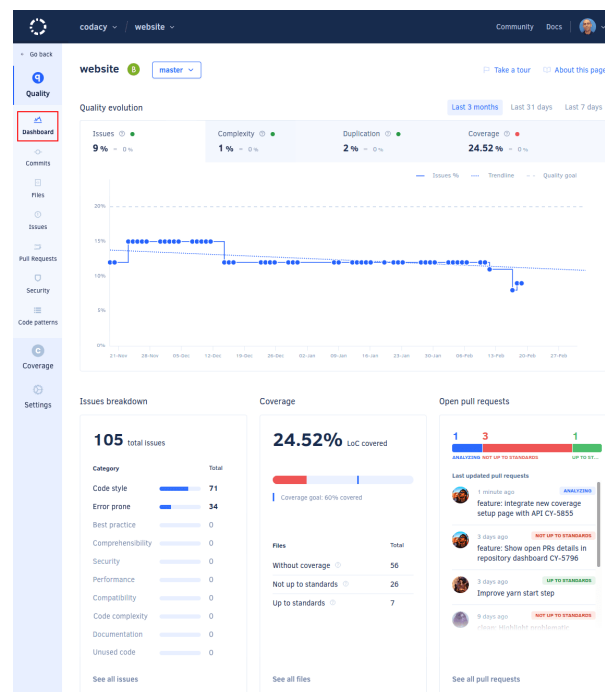
Apesar de possuir uma versão gratuita, algumas funcionalidades avançadas e recursos gerenciais são comerciais. A ferramenta também pode gerar falsos positivos e exige configuração adequada para o contexto de cada projeto. Para o escopo deste trabalho, a principal limitação é que, embora calcule complexidade ciclomática, o SonarQube apresenta

os dados principalmente em painéis e relatórios, sem priorizar a visualização gráfica e interativa do Grafo de Fluxo de Controle (GFC).

### 4.1.2 Codacy

O Codacy (Figura ??) é uma plataforma de análise estática, gratuita para projetos de código aberto, voltada à melhoria da qualidade do software, ao aumento da cobertura de testes e à prevenção de falhas de segurança ??). Em vez de se concentrar apenas em métricas matemáticas, a ferramenta prioriza a identificação prática de *bugs* e comportamentos indefinidos em diferentes linguagens de programação ??).

Figura 5 – Interface do painel de monitoramento do Codacy.



Fonte: Documentação oficial do Codacy (2026).

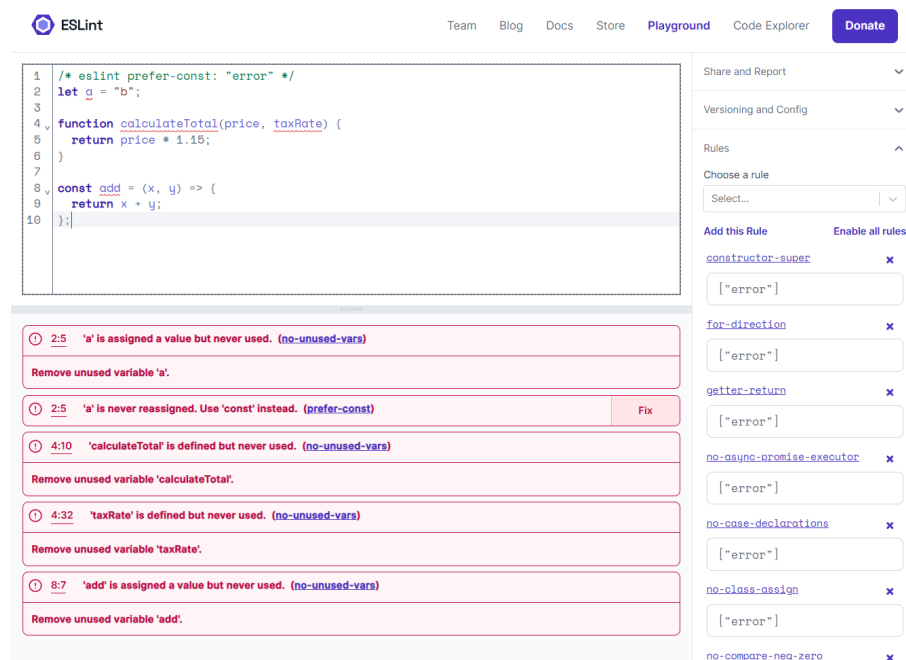
O Codacy destaca-se pela interface amigável e pela facilidade de configuração e integração aos projetos ????). Como diferencial, consolida estatísticas sobre propensão a erros, estilo e segurança em um *dashboard*, atribuindo ao projeto uma nota geral que varia de "A"(alta qualidade) a "F"(baixa qualidade) ????).

Como limitação, a agregação das métricas na nota final nem sempre deixa claro o peso de cada fator avaliado ????). A análise também depende frequentemente de *linters* de terceiros e pode gerar falsos positivos ??). Embora avalie categorias de complexidade ??), seus resultados permanecem concentrados em notas e painéis, sem fornecer uma representação gráfica clara do fluxo lógico do algoritmo analisado.

### 4.1.3 ESLint

O ESLint (Figura ??) é um *linter* plugável e uma ferramenta de análise estática amplamente utilizada, projetada para JavaScript e extensível ao TypeScript por meio de *plugins* ??????). Sua função principal é examinar o código sem executá-lo, utilizando sua representação sintática para identificar padrões, impor convenções de estilo e sinalizar erros, ajudando a manter a consistência do projeto ??). A ferramenta destaca-se por sua flexibilidade, permitindo a adoção de regras e *presets* da comunidade ??).

Figura 6 – Playground do ESLint exibindo a identificação de erros de sintaxe e estilo.



Fonte: Adaptado de ESLint (2026).

Além de oferecer correção automática (auto-fix) e um ecossistema extensível ????, o ESLint possui fácil integração com IDEs e pipelines de CI/CD, fornecendo *feedback* em tempo real e atuando como um recurso educacional para os desenvolvedores ??????).

Apesar dessas vantagens, a ferramenta possui limitações técnicas. A avaliação do ESLint é orientada principalmente à análise estática de arquivos e regras configuradas no projeto, o que limita a identificação de problemas que dependem de interações complexas entre módulos, execução real ou dependências externas ??). Sua eficácia também esbarra na natureza dinâmica do JavaScript, não sendo capaz de prever todos os erros em tempo de execução ?????). Além disso, a complexidade de configuração e a ausência de provas formais de correção podem gerar falsos positivos; esse excesso de avisos frequentemente resulta em uma “fadiga de alertas”, levando os desenvolvedores a ignorarem os reportes ou a perderem a confiança na análise ??????).

Para a análise da estrutura lógica, essa restrição é relevante. Embora possua regras para alertar quando a complexidade ciclomática de uma função excede um limite pré-estabelecido, o ESLint não tem como foco apresentar uma visualização interativa do fluxo de execução do algoritmo ??). Assim, a compreensão estrutural fica limitada a alertas e diagnósticos textuais, sem uma visualização gráfica do fluxo de controle.

## 4.2 Levantamento de Requisitos da Ferramenta GUY

Esta seção apresenta os requisitos definidos para a ferramenta GUY.

O acesso à ferramenta ocorre diretamente no ambiente de desenvolvimento, por meio de uma extensão para o editor *Visual Studio Code* (VS Code) ??). Um requisito fundamental é que a análise estática seja executada localmente. Para isso, a proposta utiliza tecnologias como *WebAssembly* e *Tree-sitter*, responsáveis por apoiar a construção da árvore sintática do código analisado.

Essa arquitetura mantém o processamento léxico e sintático no ambiente do usuário e evita o envio do código-fonte para servidores externos. Com isso, a extensão reduz riscos relacionados à privacidade e à propriedade intelectual, além de dispensar infraestrutura de nuvem para executar a análise.

No aspecto funcional, a principal entrega do sistema é a renderização visual interativa do Grafo de Fluxo de Controle (GFC) após a análise do código-fonte. A interface deve permitir que o programador interaja com o grafo gerado, relacionando nós de decisão e arestas direcionadas às linhas correspondentes do arquivo aberto no editor. Em conjunto com essa visualização, a extensão deve calcular e exibir a métrica de Complexidade Ciclômica, usada como indicador da complexidade lógica e da testabilidade do código.

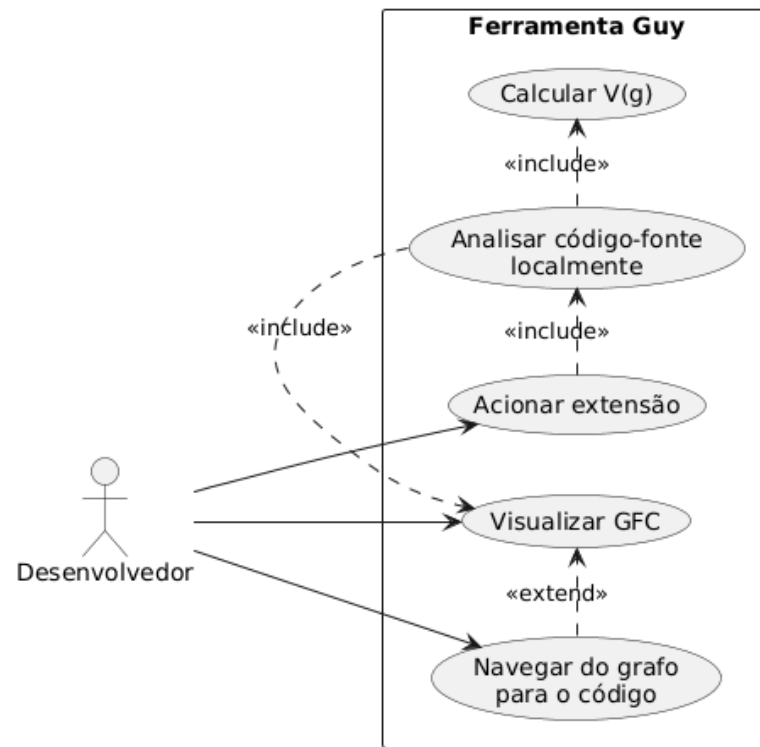
Para apoiar a análise da qualidade estrutural do software, a extensão também deve indicar caminhos linearmente independentes no grafo. Esses dados auxiliam o programador na elaboração de casos de teste alinhados aos fluxos identificados.

## 4.3 Prototipagem e *Design* da Ferramenta GUY

O usuário aciona a extensão com o editor de código aberto e em foco. Em resposta, a ferramenta abre uma janela lateral, de forma semelhante a uma pré-visualização de Markdown, apresentando o Grafo de Fluxo de Controle (GFC) e o cálculo da Complexidade Ciclômica, representada por  $V(G)$ . Nessa visualização, o usuário pode selecionar vértices e arestas do grafo para navegar diretamente até os trechos correspondentes do código-fonte, estabelecendo uma relação interativa entre a representação visual e a implementação analisada.

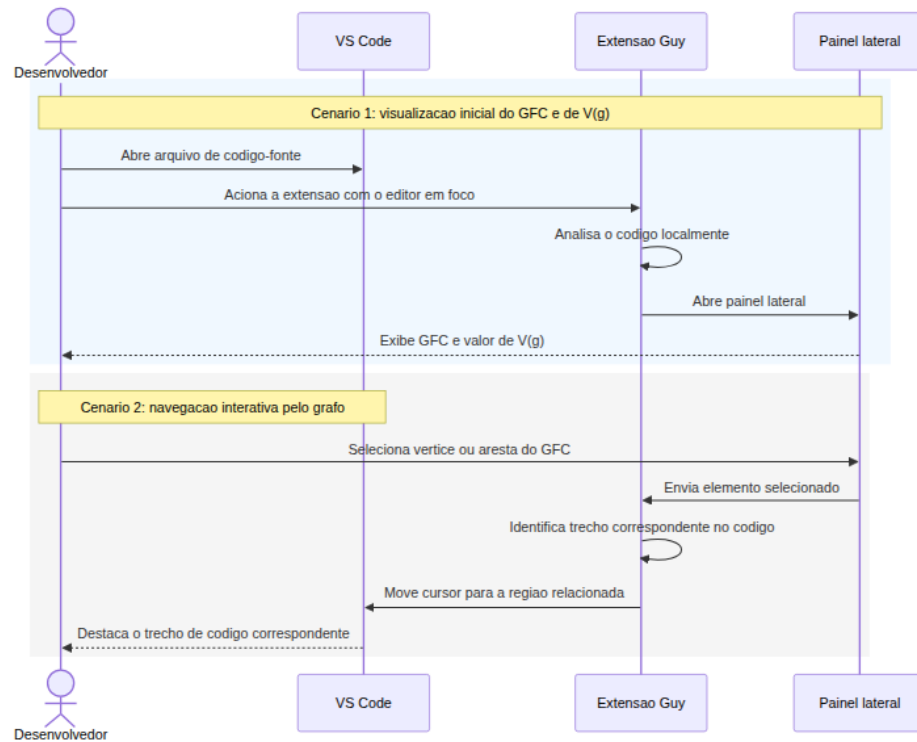
Essa proposta atende aos principais requisitos definidos para a ferramenta: execução local, sem envio do código-fonte para servidores externos; apresentação do GFC em conjunto com o valor de  $V(G)$ ; e interação direta entre os elementos do grafo e as regiões correspondentes do código.

Figura 7 – Diagrama de casos de uso da Ferramenta GUY.



Fonte: Elaborada pelo autor.

Figura 8 – Diagrama de cenários (sequência) da Ferramenta GUY.



Fonte: Elaborada pelo autor.

## 4.4 Implementação da Ferramenta GUY

A ferramenta foi desenvolvida em *TypeScript* como uma extensão para o editor de código VS Code, com o código completo disponível no GitHub em <<https://github.com/gm64x/GUY-vscode>>. Seu objetivo é integrar a análise de fluxo de controle diretamente ao ambiente de desenvolvimento. O nome GUY deriva de *Graphing Utility for Your code* e, em sua versão atual, tem como foco a análise de arquivos Python.

A extensão utiliza a API do VS Code para registrar comandos, acessar o editor ativo e abrir a visualização lateral por meio de uma *Webview*. Nessa interface, o desenvolvedor pode navegar entre os elementos do grafo e as linhas correspondentes do código-fonte. A interface visual do GFC foi construída com *React* e *@xyflow/react*, enquanto a organização automática dos nós utiliza a biblioteca *@dagrejs/dagre*. Além disso, a análise sintática do código Python é realizada pelo *Tree-sitter* por meio de *WebAssembly*.

A configuração inicial da extensão está disponível no arquivo `package.json`, que é lido pelo próprio editor. Esse arquivo declara quando a extensão deve ser ativada, quais comandos estão disponíveis para o usuário e em quais menus eles aparecem. O trecho a seguir mostra a parte relacionada aos comandos principais da extensão.

Listing 4.1 – Recorte da configuração de comandos da extensão GUY.

```
1 "activationEvents": [
```

```
2  "onLanguage:python",
3  "onCommand:guy.generateCfgFromFile",
4  "onCommand:guy.generateCfgFromSelection",
5  "onCommand:guy.generateCfgFromCurrentFunction",
6  "onCommand:guy.toggleGraphDetailMode"
7 ],
8 "contributes": {
9   "commands": [
10    {
11      "command": "guy.generateCfgFromFile",
12      "title": "Generate CFG from File",
13      "category": "GUY"
14    },
15    {
16      "command": "guy.generateCfgFromSelection",
17      "title": "Generate CFG from Selection",
18      "category": "GUY"
19    },
20    {
21      "command": "guy.generateCfgFromCurrentFunction",
22      "title": "Generate CFG from Current Function",
23      "category": "GUY"
24    },
25    {
26      "command": "guy.toggleGraphDetailMode",
27      "title": "Toggle Simplified/Detailed CFG View",
28      "category": "GUY"
29    }
30  ]
31 }
```

O mesmo arquivo também define opções de uso da extensão, como abertura automática da pré-visualização, direção do layout do grafo, exibição do painel de métricas, destaque de código ao selecionar nós e limite considerado alto para a Complexidade Ciclomática.

#### 4.4.1 Estrutura geral da extensão

A implementação está organizada em módulos com responsabilidades separadas. O arquivo `src/extension.ts` registra os comandos da extensão e coordena o fluxo principal de execução. Os módulos do diretório `src/core/` concentram a análise do código-fonte, a construção do GFC, o cálculo das métricas e a identificação de caminhos independentes. Já os módulos em `src/vscode/` tratam da integração com o editor, incluindo a criação da janela lateral e o destaque de trechos do código. A interface React da pré-visualização fica em `src/webview/`.

A Tabela ?? resume os módulos mais importantes da implementação.

Tabela 1 – Principais módulos da implementação da ferramenta GUY.

| Módulo                                                           | Responsabilidade                                                                                                                                                                        |
|------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>extension.ts</code>                                        | Registra os comandos da extensão, obtém o arquivo ativo, controla o modo de geração do grafo e envia os resultados para a Webview.                                                      |
| <code>parser.ts</code>                                           | Utiliza o Tree-sitter para transformar o código Python em uma estrutura sintática intermediária, identificando funções, comandos sequenciais, condicionais, laços e comandos de desvio. |
| <code>cfgBuilder.ts</code>                                       | Constrói os nós e as arestas do Grafo de Fluxo de Controle a partir da estrutura sintática gerada pelo parser.                                                                          |
| <code>metrics.ts</code>                                          | Calcula a quantidade de nós, arestas, decisões, componentes conexos e a Complexidade Ciclomática.                                                                                       |
| <code>paths.ts</code>                                            | Lista caminhos independentes entre o nó de entrada e o nó de saída, respeitando limites para grafos grandes.                                                                            |
| <code>webviewPanel.ts</code><br><code>webview/src/App.tsx</code> | e Cria a pré-visualização interativa do grafo, a barra lateral de métricas, as listas de nós, arestas, funções e caminhos independentes.                                                |
| <code>editorNavigation.ts</code>                                 | Realiza o destaque e a navegação entre elementos selecionados no grafo e os trechos correspondentes do arquivo Python.                                                                  |

Fonte: Elaborada pelo autor.

Essa divisão separa a análise do código Python da integração com o editor. O diretório `src/core/` concentra a construção do grafo e das métricas, enquanto `src/vscode/` e `src/webview/` cuidam da comunicação com o VS Code e da interface visual.

#### 4.4.2 Comandos e formas de análise

A extensão disponibiliza comandos no VS Code para gerar o GFC a partir de três escopos de análise: arquivo completo, trecho selecionado e função atual. Esses comandos podem ser acionados pela Paleta de Comandos e também por menus de contexto do editor Python. O comando de arquivo completo fica disponível na barra de título do editor, enquanto os comandos de seleção e função atual ficam disponíveis no menu de contexto.

Tabela 2 – Comandos principais da ferramenta GUY.

| Comando                                  | Descrição                                                            |
|------------------------------------------|----------------------------------------------------------------------|
| GUY: Generate CFG from File              | Gera o GFC a partir de todo o arquivo Python ativo.                  |
| GUY: Generate CFG from Selection         | Gera o GFC apenas para o trecho de código selecionado no editor.     |
| GUY: Generate CFG from Current Function  | Gera o GFC da função Python na qual o cursor está posicionado.       |
| GUY: Toggle Simplified/Detailed CFG View | Alterna a visualização entre o modo simplificado e o modo detalhado. |

Fonte: Elaborada pelo autor.

O fluxo de execução parte do editor ativo. Ao executar um comando, a extensão verifica se o documento é Python, captura o conteúdo correspondente ao escopo escolhido e aciona o construtor de grafos. Após a construção do GFC, os dados são enviados para a Webview, que renderiza a visualização em uma coluna lateral do VS Code.

No ponto de entrada da extensão, em `src/extension.ts`, a função `activate` instancia o construtor do grafo, o navegador do editor e o painel da Webview. Em seguida, associa cada comando declarado no `package.json` à respectiva função TypeScript.

Listing 4.2 – Registro dos comandos principais da extensão.

```

1 export function activate(context: vscode.ExtensionContext): void {
2   const builder = new CFGBuilder();
3   navigator = new EditorNavigator();
4   panel = new GuyWebviewPanel(context.extensionUri, (message) =>
5     handleWebviewMessage(message, builder),
6   );
7
8   context.subscriptions.push(
9     navigator,
10    vscode.commands.registerCommand("guy.generateCfgFromFile", () =>
11      generateFromFile(builder),
12    ),
13    vscode.commands.registerCommand("guy.generateCfgFromSelection", ()
14      =>
15        generateFromSelection(builder),
16    ),
17    vscode.commands.registerCommand("guy.generateCfgFromCurrentFunction"
18      , () =>
19        generateFromCurrentFunction(builder),
20    ),
21    vscode.commands.registerCommand("guy.toggleGraphDetailMode", () =>
22      toggleDetailMode(builder),
23    ),
24  );

```

---

23 }

---

Para a análise do arquivo completo, a extensão lê o conteúdo do documento ativo, chama `builder.generate` e envia o resultado para o painel lateral.

Listing 4.3 – Geração do GFC a partir do arquivo Python ativo.

---

```
1 async function generateFromFile(builder: CFGBuilder): Promise<void> {
2   const editor = getPythonEditor();
3   if (!editor) {
4     return;
5   }
6
7   panel.loading();
8   currentCfg = await builder.generate({
9     source: editor.document.getText(),
10    fileName: editor.document.fileName,
11    mode: "file",
12    viewMode: currentViewMode,
13    highComplexityThreshold: getHighComplexityThreshold(),
14  });
15  panel.show(currentCfg);
16  showDiagnostics(currentCfg);
17 }
```

---

#### 4.4.3 Construção do Grafo de Fluxo de Controle

A análise do código é realizada em duas etapas principais. Primeiro, o parser baseado em Tree-sitter percorre a árvore sintática do código Python e classifica os elementos relevantes para o fluxo de controle. Na versão atual, são tratados comandos sequenciais, funções, estruturas `if/elif/else`, laços `for` e `while`, além de comandos `return`, `break` e `continue`. Esses elementos são representados internamente pelos tipos indicados no trecho a seguir.

Listing 4.4 – Tipos de comandos Python reconhecidos pelo parser.

---

```
1 export type PythonStatementKind =
2   | "statement"
3   | "if"
4   | "loop"
5   | "return"
6   | "break"
7   | "continue"
8   | "function";
```

---

Durante o processamento, cada nó relevante da árvore sintática é convertido em uma estrutura interna da ferramenta GUY. Definições de função, condicionais, laços e comandos de desvio são diferenciados pelo tipo retornado pelo Tree-sitter.

Listing 4.5 – Recorte simplificado da classificação de nós sintáticos relevantes para o GFC.

---

```

1 private nodeFromTreeSitter(
2   node: TreeSitterNode,
3   offset: SourceOffset,
4 ): PythonAstNode | undefined {
5   if (node.type === "function_definition") {
6     return { kind: "function", name, code: firstLine(node.text), body };
7   }
8
9   if (node.type === "if_statement") {
10    return this.ifFromTreeSitter(node, offset);
11  }
12
13  if (node.type === "for_statement" || node.type === "while_statement")
14    {
15    return { kind: "loop", loopKind, condition, code, body };
16  }
17
18  if (node.type === "return_statement") {
19    return { kind: "return", code: node.text };
20  }

```

---

Depois da etapa de análise sintática, o módulo `cfgBuilder.ts` transforma a estrutura intermediária no GFC. Todo grafo gerado possui um nó inicial, `Entry`, e um nó final, `Exit`. Comandos sequenciais usam arestas `next`; condicionais usam arestas `true` e `false`; laços usam arestas de repetição `loop`; e retornos são conectados diretamente ao nó de saída.

Listing 4.6 – Fluxo principal de construção do GFC.

---

```

1 const entry = this.addNode("Entry", "entry", "", defaultRange);
2 const exit = this.addNode("Exit", "exit", "", body[body.length - 1]);
3 this.exitNodeId = exit.id;
4
5 const open = this.buildSequence(
6   body,
7   [entry.id],
8   undefined,
9   { loopStack: [] },
10  this.options.viewMode,
11 );
12

```

---

---

```

13 for (const from of open) {
14   this.addEdge(from, exit.id, "next");
15 }
16
17 const metrics = calculateMetrics(this.nodes, this.edges);
18 const independentPathResult = calculateIndependentPaths(
19   this.nodes,
20   this.edges,
21   entry.id,
22   exit.id,
23   metrics.cyclomaticComplexity,
24 );

```

---

Nas condicionais, a ferramenta cria um nó para a condição e um nó de junção, chamado **merge**, para reunir os ramos ao final da estrutura. O ramo verdadeiro recebe a aresta **true**; o fluxo alternativo segue pela aresta **false**.

Listing 4.7 – Recorte da construção de estruturas condicionais.

---

```

1  const condition = this.addNode(
2    'if ${branch.condition ?? ""}',
3    "condition",
4    branch.condition ?? statement.code,
5    branch,
6  );
7
8  this.connect(pendingFalseFrom, condition.id, pendingLabel);
9  const branchExits = this.buildSequence(
10    branch.body,
11    [condition.id],
12    "true",
13    context,
14    viewMode,
15  );
16
17 pendingFalseFrom = [condition.id];
18 pendingLabel = "false";
19
20 const merge = this.addNode("merge", "merge", "", statement);
21 this.connect(pendingFalseFrom, merge.id, pendingLabel);
22 this.connect(exits, merge.id, "next");

```

---

Nos laços, a ferramenta cria um nó para a condição do **for** ou **while**, um nó de saída e uma aresta de retorno para representar a repetição.

Listing 4.8 – Recorte da construção de laços no GFC.

---

```

1  const condition = this.addNode(
2    '${statement.loopKind} ${statement.condition}',

```

---

```

3   "loop",
4   statement.code,
5   statement,
6 );
7 const afterLoop = this.addNode("after loop", "merge", "", statement);
8
9 this.connect(predecessors, condition.id, firstLabel ?? "next");
10 const bodyExits = this.buildSequence(
11   statement.body,
12   [condition.id],
13   "true",
14   loopContext,
15   viewMode,
16 );
17
18 this.connect(bodyExits, condition.id, "loop");
19 this.addEdge(condition.id, afterLoop.id, "false");

```

---

No caso de laços, a ferramenta mantém uma pilha interna para associar corretamente comandos `break` e `continue` ao laço em que aparecem. Com isso, um `break` é ligado ao ponto posterior ao laço, enquanto um `continue` retorna para a condição do laço. Esse controle é necessário porque esses comandos dependem do laço mais interno em execução, e não apenas da posição textual em que aparecem.

A ferramenta também oferece dois modos de visualização. No modo simplificado, comandos sequenciais adjacentes são compactados em um único nó, reduzindo a densidade visual do grafo. No modo detalhado, os comandos são preservados de forma mais granular, favorecendo uma análise mais próxima da estrutura linha a linha do código.

#### 4.4.4 Métricas e caminhos independentes

Após a construção do grafo, a ferramenta calcula métricas estruturais do GFC. A principal delas é a Complexidade Ciclomática, calculada pela expressão:

$$V(G) = E - N + 2P \quad (4.1)$$

em que  $E$  representa o número de arestas,  $N$  o número de nós e  $P$  o número de componentes conexos do grafo. A extensão também apresenta a quantidade de decisões, o total de nós e arestas e a lista de caminhos independentes entre o nó de entrada e o nó de saída. O cálculo fica concentrado no módulo `metrics.ts`.

---

Listing 4.9 – Cálculo das métricas estruturais do GFC.

---

```

1 export function calculateMetrics(nodes: CFGNode[], edges: CFGEdge[]):
  CFGMetrics {

```

---

```

2  const decisionCount = nodes.filter(
3    (node) => node.kind === "condition" || node.kind === "loop",
4  ).length;
5  const connectedComponents = countConnectedComponents(nodes, edges);
6  const cyclomaticComplexity = Math.max(
7    1,
8    edges.length - nodes.length + 2 * connectedComponents,
9  );
10
11  return {
12    nodeCount: nodes.length,
13    edgeCount: edges.length,
14    decisionCount,
15    connectedComponents,
16    cyclomaticComplexity,
17    simplifiedCyclomaticComplexity: decisionCount + 1,
18  };
19 }

```

---

A interface também possui um limite de complexidade configurável. Por padrão, quando  $V(G)$  ultrapassa o valor 10, a extensão exibe sugestões simples, indicando que a função pode ser candidata a refatoração, divisão em funções menores ou redução de condicionais aninhadas.

#### 4.4.5 Interface de visualização

A visualização do GFC é apresentada em uma Webview lateral. O grafo é renderizado com nós, arestas direcionadas e rótulos que indicam o tipo de transição, como `true`, `false`, `loop`, `return`, `break` e `continue`. A interface oferece controles para ajustar a visualização, centralizar o grafo, redefinir o zoom e ampliar ou reduzir a escala.

A barra lateral da Webview apresenta as métricas calculadas, a fórmula de Complexidade Ciclômática aplicada ao grafo gerado, a lista de nós, a lista de arestas, as funções encontradas e os caminhos independentes. Ao selecionar um nó, uma aresta ou um caminho, a extensão envia uma mensagem da Webview para o VS Code e destaca o trecho correspondente no editor. Essa interação atende ao objetivo educacional da ferramenta, pois permite relacionar diretamente a estrutura visual do fluxo de controle com o código-fonte analisado.

A janela lateral é criada no módulo `webviewPanel.ts`. O painel é aberto ao lado do editor, permite execução de scripts e registra o canal de mensagens usado pela interface React.

Listing 4.10 – Criação da Webview lateral no VS Code.

---

```

1  this.panel = vscode.window.createWebviewPanel(

```

---

```

2  "guyCfgPreview",
3  title,
4  vscode.ViewColumn.Beside,
5  {
6    enableScripts: true,
7    retainContextWhenHidden: true,
8    localResourceRoots: [this.extensionUri],
9  },
10 );
11
12 this.panel.webview.onDidReceiveMessage((message: WebviewMessage) =>
13   this.onMessage(message),
14 );
15 this.panel.webview.html = getWebviewHtml(
16   this.panel.webview,
17   this.extensionUri,
18   cfg,
19 );

```

---

Na interface React, os nós e arestas do GFC são convertidos para o formato esperado pelo `@xyflow/react`. A biblioteca `@dagrejs/dagre` calcula a posição automática dos elementos, e os eventos de clique enviam mensagens para a extensão.

Listing 4.11 – Renderização interativa do GFC na Webview.

---

```

1  <ReactFlow
2    nodes={flowNodes}
3    edges={flowEdges}
4    nodeTypes={nodeTypes}
5    onNodeClick={({_, node}) => selectNode(node.id)}
6    onEdgeClick={({_, edge}) => selectEdge(edge.id)}
7    nodesConnectable={false}
8    elementsSelectable
9  >
10   <Background color="var(--border)" gap={20} size={0.5} />
11 </ReactFlow>
12
13 const selectNode = useCallback((nodeId: string) => {
14   setSelectedNodeId(nodeId);
15   vscode?.postMessage({ type: "NODE_SELECTED", payload: { nodeId } });
16 }, []);

```

---

O módulo `editorNavigation.ts` completa essa integração: quando recebe uma seleção vinda da Webview, localiza o trecho correspondente no arquivo Python e aplica uma decoração temporária no editor. Assim, grafo e código permanecem sincronizados.

#### 4.4.6 Critérios de implementação observados

Os pontos técnicos verificados na implementação aparecem em arquivos específicos da extensão. A Tabela ?? relaciona cada critério com a evidência encontrada no código-fonte.

Tabela 3 – Critérios técnicos observados na implementação da ferramenta GUY.

| Critério                              | Evidência na implementação                                                                                                                             |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Comandos, menus e eventos de ativação | Declarados no <code>package.json</code> , por meio de <code>activationEvents</code> , <code>contributes.commands</code> e <code>menus</code> .         |
| Registro dos comandos                 | Realizado em <code>src/extension.ts</code> , dentro da função <code>activate</code> , com chamadas a <code>vscode.commands.registerCommand</code> .    |
| Uso real do Tree-sitter               | Implementado em <code>src/core/parser.ts</code> , com <code>web-tree-sitter</code> e a gramática <code>tree-sitter-python</code> .                     |
| Ausência de fallback textual          | Quando o Tree-sitter não carrega, a ferramenta registra diagnóstico e não tenta reconstruir o grafo com um parser simplificado.                        |
| Estrutura do GFC                      | Definida em <code>src/core/types.ts</code> , com nós, arestas, <code>entryNodeId</code> , <code>exitNodeId</code> , métricas e caminhos independentes. |
| Posições do código original           | Calculadas a partir das posições do Tree-sitter e do deslocamento da seleção, mantendo linha e coluna do arquivo aberto.                               |
| Integração grafo-código               | Cliques na Webview enviam mensagens para a extensão, que destaca os trechos correspondentes no editor.                                                 |
| Integração código-grafo               | Mudanças de seleção no editor procuram o nó correspondente e o destacam na Webview.                                                                    |
| Tema claro e escuro                   | A Webview usa variáveis CSS nativas do VS Code, como <code>-vscode-editor-background</code> e <code>-vscode-editor-foreground</code> .                 |

Fonte: Elaborada pelo autor.

Um ponto importante da implementação é que o parser não possui fallback textual. A ferramenta depende do Tree-sitter para montar a estrutura sintática usada na geração do GFC. Se o WebAssembly do parser não puder ser carregado, a extensão retorna um diagnóstico e evita produzir um grafo baseado em inferências frágeis.

Listing 4.12 – Tratamento de falha no carregamento do Tree-sitter.

```

1 try {
2   const runtime = await this.getWebTreeSitter();
3   const parser = new runtime.Parser();
4   parser.setLanguage(runtime.python);

```

---

```

5   const tree = parser.parse(source);
6   const body = this.nodesFromTreeSitterContainer(tree.rootNode, offset);
7   return { body, functions, diagnostics, treeSitterAvailable: true };
8 } catch (err: any) {
9   diagnostics.push(
10    'Tree-sitter WASM could not be loaded: ${err?.message ?? err}',
11  );
12  return { body: [], functions: [], diagnostics, treeSitterAvailable:
13    false };

```

---

As posições de origem dos nós são preservadas pela função que converte os intervalos retornados pelo Tree-sitter. Quando a análise é feita sobre uma seleção, o deslocamento inicial da seleção é somado às posições relativas do parser. Por isso, o destaque no editor aponta para as linhas reais do arquivo, e não apenas para o trecho isolado analisado.

Listing 4.13 – Conversão das posições do Tree-sitter para o arquivo original.

---

```

1  function rangeFromTreeSitter(
2    node: TreeSitterNode,
3    offset: SourceOffset,
4  ): SourceRange {
5    return {
6      startLine: offset.line + node.startPosition.row,
7      startColumn:
8        node.startPosition.row === 0
9        ? offset.column + node.startPosition.column
10       : node.startPosition.column,
11      endLine: offset.line + node.endPosition.row,
12      endColumn:
13        node.endPosition.row === 0
14        ? offset.column + node.endPosition.column
15       : node.endPosition.column,
16    };
17  }

```

---

A sincronização também funciona no sentido contrário. Quando o cursor muda de posição no editor, a extensão procura o menor nó cujo intervalo contém a posição atual. Se encontrar esse nó, envia uma mensagem para a Webview destacá-lo no grafo.

Listing 4.14 – Seleção no editor destacando o nó correspondente.

---

```

1  vscode.window.onDidChangeTextEditorSelection((event) => {
2    if (event.textEditor !== vscode.window.activeTextEditor) {
3      return;
4    }
5    if (navigator.isSelectionFromNavigation()) {
6      return;

```

---

```

7   }
8   const node = navigator.findNodeAt(currentCfg, event.selections[0].
      active);
9   if (node) {
10    panel.highlightNode(node.id);
11  }
12 });

```

---

A adaptação ao tema do VS Code fica no CSS da Webview. Em vez de fixar cores próprias, a interface utiliza variáveis fornecidas pelo editor. Assim, o fundo, o texto, as bordas e os destaques acompanham o tema claro ou escuro escolhido pelo usuário.

Listing 4.15 – Uso de variáveis de tema do VS Code na Webview.

---

```

1 :root {
2   --bg: var(--vscode-editor-background, #1e1e1e);
3   --fg: var(--vscode-editor-foreground, #cccccc);
4   --muted: var(--vscode-descriptionForeground, #858585);
5   --accent: var(--vscode-focusBorder, #007acc);
6   --border: var(--vscode-panel-border, #3c3c3c);
7   --surface: var(--vscode-sideBar-background, #252526);
8 }

```

---

## 4.5 Validação da Ferramenta GUY

A validação inicial da ferramenta foi realizada com os arquivos de exemplo disponíveis em `app/src/examples/`. Os experimentos têm como objetivo verificar se a extensão representa corretamente diferentes estruturas de controle em Python e se as métricas exibidas são coerentes com o grafo construído. Todos os resultados apresentados nesta seção foram obtidos no modo de visualização simplificado.

### 4.5.1 Experimento I: Código com Laço e Condicionais

O primeiro experimento utiliza o arquivo `loop_break_continue.py`. O objetivo é verificar se a ferramenta representa corretamente um laço `while` contendo duas estruturas condicionais internas, uma com `continue` e outra com `break`. Esse cenário é importante porque envolve desvios que alteram o fluxo normal da iteração.

Listing 4.16 – Código utilizado no Experimento I.

---

```

1 x = 0
2
3 while x < 10:
4     x += 1
5     if x == 3:

```

---

```

6         continue
7     if x == 8:
8         break
9
10 print(x)

```

---

Ao analisar esse código, a ferramenta GUY identifica três pontos de decisão: a condição do `while` e os dois comandos `if`. O comando `continue` é representado por uma aresta que retorna para a condição do laço, enquanto o comando `break` é representado por uma aresta que conduz ao ponto posterior ao laço.

Tabela 4 – Métricas obtidas no Experimento I.

| Nós | Arestas | Decisões | Componentes | $V(G)$ | Caminhos independentes |
|-----|---------|----------|-------------|--------|------------------------|
| 13  | 15      | 3        | 1           | 4      | 4                      |

Fonte: Elaborada pelo autor.

A Complexidade Ciclomática resultante é  $V(G) = 15 - 13 + 2 \times 1 = 4$ . Esse valor é coerente com a quantidade de decisões presentes no código, pois, para um único componente conexo, a expressão geral simplifica-se para  $V(G) = E - N + 2$ . A complexidade também pode ser interpretada como o número de decisões acrescido de uma unidade.

#### 4.5.2 Experimento II: Código Condicional Simples

O segundo experimento utiliza o arquivo `conditional.py`. O objetivo é validar o comportamento da ferramenta diante de uma estrutura condicional simples, composta por um `if`, um `else` e uma instrução executada após a junção dos dois ramos.

Listing 4.17 – Código utilizado no Experimento II.

---

```

1 x = 10
2
3 if x > 5:
4     print("maior")
5 else:
6     print("menor")
7
8 print("fim")

```

---

Nesse caso, a ferramenta cria um nó de decisão para a condição `x > 5`, duas saídas para representar os ramos verdadeiro e falso e um nó de junção antes da continuidade do fluxo para o comando final.

Tabela 5 – Métricas obtidas no Experimento II.

| Nós | Arestas | Decisões | Componentes | $V(G)$ | Caminhos independentes |
|-----|---------|----------|-------------|--------|------------------------|
| 8   | 8       | 1        | 1           | 2      | 2                      |

Fonte: Elaborada pelo autor.

A Complexidade Ciclomática calculada é  $V(G) = 8 - 8 + 2 \times 1 = 2$ , representando os dois fluxos possíveis de execução: o caminho em que a condição é verdadeira e o caminho em que ela é falsa.

### 4.5.3 Experimento III: Código com Função, Laço e Condicional

O terceiro experimento utiliza o arquivo `functions.py`, analisando especificamente a função `calculate_total`. O objetivo é verificar o comando de geração do GFC a partir da função atual, bem como a representação combinada de laço `for`, condicional `if` e retorno de valor.

Listing 4.18 – Código utilizado no Experimento III.

```

1 def calculate_total(items):
2     total = 0
3     for item in items:
4         if item > 0:
5             total += item
6     return total

```

Ao posicionar o cursor dentro da função e executar o comando **GUY: Generate CFG from Current Function**, a extensão restringe a análise ao corpo da função selecionada. O grafo resultante contém a inicialização da variável, a condição de iteração do `for`, a decisão interna do `if`, o retorno ao laço e o comando `return` conectado ao nó de saída.

Tabela 6 – Métricas obtidas no Experimento III.

| Nós | Arestas | Decisões | Componentes | $V(G)$ | Caminhos independentes |
|-----|---------|----------|-------------|--------|------------------------|
| 9   | 10      | 2        | 1           | 3      | 3                      |

Fonte: Elaborada pelo autor.

A Complexidade Ciclomática calculada é  $V(G) = 10 - 9 + 2 \times 1 = 3$ . O resultado reflete os dois pontos de decisão presentes na função: a repetição do `for` e a condição `if item > 0`. Assim, a ferramenta fornece uma indicação objetiva da quantidade mínima de caminhos independentes que devem ser considerados em uma estratégia de teste baseada em fluxo de controle.

## 4.6 Análise Qualitativa da Ferramenta GUY

A análise qualitativa do protótipo indica que o GUY atende ao objetivo central do trabalho: apoiar a compreensão do fluxo lógico de programas por meio de uma representação visual interativa do GFC. Diferentemente de ferramentas voltadas principalmente a painéis de qualidade ou diagnósticos textuais, o GUY relaciona a métrica de Complexidade Ciclomática à estrutura concreta do código analisado.

Entre os principais pontos positivos, destaca-se a integração direta com o VS Code. O desenvolvedor não precisa copiar o código para uma ferramenta externa, pois a análise é acionada no próprio editor. Além disso, o processamento local contribui para a privacidade do código-fonte e reduz a dependência de serviços externos. A possibilidade de gerar grafos a partir do arquivo inteiro, de uma seleção ou da função atual também torna a ferramenta flexível para diferentes contextos de uso.

Outro aspecto positivo é a interação entre grafo e código-fonte. A seleção de nós, arestas e caminhos independentes na Webview destaca as regiões correspondentes no editor, favorecendo o entendimento da relação entre a sintaxe do programa e seu fluxo de execução. A alternância entre os modos simplificado e detalhado também é relevante: o modo simplificado reduz a complexidade visual em exemplos maiores, enquanto o modo detalhado permite uma inspeção mais granular.

Como limitação, a versão atual concentra-se em Python e em estruturas de controle comuns, como condicionais, laços, retornos, **break** e **continue**. Casos mais avançados da linguagem, como exceções, chamadas entre funções, recursão, geradores, programação assíncrona e efeitos dinâmicos de execução, ainda não são representados de forma completa. Além disso, por se tratar de análise estática, a ferramenta não substitui a execução de testes nem garante a identificação de todos os comportamentos possíveis em tempo de execução.

Também há limitações relacionadas à visualização de grafos muito grandes. Embora a interface apresente avisos e o sistema oculte a listagem de caminhos independentes em grafos extensos para manter a responsividade, programas com muitas estruturas aninhadas podem gerar representações visualmente densas. Nesses casos, a análise por seleção ou por função tende a ser mais adequada do que a análise do arquivo completo.

Os experimentos indicam que o GUY gera grafos coerentes para estruturas fundamentais de controle de fluxo e apresenta métricas compatíveis com a teoria da Complexidade Ciclomática. Como trabalhos futuros, podem ser considerados o suporte a mais estruturas da linguagem Python, a exportação dos grafos para formatos externos, a inclusão de testes automatizados com exemplos de referência e a ampliação da ferramenta para outras linguagens de programação.

## 5 CONCLUSÕES E RECOMENDAÇÕES

São descritas claramente as conclusões retiradas das discussões e dos experimentos realizados no decorrer da pesquisa, e finalizada a parte textual do trabalho. Recomendações são declarações concisas de ações, julgadas necessárias a partir das conclusões obtidas, a serem usadas no futuro. Ou seja, lembre-se de apresentar os possíveis trabalhos futuros derivados do seu trabalho.